3차 해커톤 상세 매뉴얼

리버스 엔지니어링 | 깃허브 툴 적용하기 | API 활용하기 | 텔레그램 봇 알림

학회원 배포용 · 2026년 10월 · 김범준

이 매뉴얼을 읽는 법

이 문서는 3차 해커톤의 네 가지 핵심 조건을 **처음 듣는 사람도 혼자 따라 할 수 있게** 적은 것입니다. 어려운 단어는 처음 나올 때마다 쉬운 말로 다시 풀었고, 모든 절차는 윈도우 노트북에서 Cursor와 터미널을 쓰는 우리 학회 방식 기준입니다.

네 키워드가 하나로 이어지는 순서

① 뜯는다
리버스 엔지니어링
원본 앱의 설계도를 복원

→

② 도구로 뜯는다
깃허브 툴
손으로 못 하는 걸 도구가

→

③ 받는다
공개 API
원본이 쓰는 데이터를 직접

→

④ 알린다
텔레그램 봇
혼자 감시하다 폰으로

①에서 "이 앱의 이 값은 어디서 오나"를 알아내야 ③에서 뭘 받을지 정해지고, ③이 돼야 ④가 감시할 대상이 생깁니다. ②는 ①을 빠르게 해주는 손입니다. **순서를 바꾸지 마세요.** 특히 API 목록부터 뒤지고 앱을 나중에 고르면, 해부가 형식이 됩니다.

이번 해커톤 조건 한눈에

조건	충족 기준
① 뜯는다	운영진 도구 목록에서 1개 이상을 써서 원본을 해부하고, 그 산출물을 plan.md '원본 해부' 섹션에 넣는다. 앱 파일은 열지 않는다.
② 공개 API로 받는다	결과물의 데이터는 공식 공개 API 또는 로그인 없는 공개 페이지에서만. API 이름·한도·호출 간격을 plan.md에 적는다.
③ 혼자 알린다	아무도 링크를 열지 않아도 정해진 간격마다 확인하고, 조건에 맞을 때만 텔레그램으로 보낸다. 같은 알림이 반복되면 안 된다.
④ 비틀기 1개	원본에 없는 기능 하나. 원본을 통째로 다시 만들지 않는다.
⑤ 선을 지킨다	본인 계정·본인 화면만, 읽기만, 비공개 주소 호출 금지, 원본 로고 사용 금지. 1장 끝에 자세히.

자주 나오는 말 — 쉬운 말 사전

말	뜻
리버스 엔지니어링	완성품을 보고 설계도를 거꾸로 그려내는 일. 이 문서에서는 줄여서 **"뜯는다"**고 씁니다.
API	어떤 서비스가 "이 주소로 이렇게 물어보면 데이터를 돌려주겠다"고 열어둔 창구. 사람이 보는 창구가 웹사이트라면, API는 **프로그램이 보는 창구**입니다.
공개 API	누구나 쓰라고 설명서와 열쇠(키)를 나눠주는 창구. 공공데이터포털이 대표적.
비공개 API	그 회사 앱만 쓰라고 만든 창구. 설명서가 없고 약관으로 금지. 우리는 안 씁니다.
키 / 토큰	창구에 들어갈 때 보여주는 열쇠. 남에게 보여주면 안 되고, 코드에 직접 적으면 안 됩니다.

JSON	프로그램끼리 데이터를 주고받을 때 쓰는 글자 형식. 중괄호와 따옴표로 된 표라고 생각하면 됩니다.
네트워크 탭	크롬 개발자 도구 안에서, 지금 이 페이지가 어디에 무엇을 요청하는지 보여주는 창. F12로 엽니다.
HAR 파일	네트워크 탭에서 본 요청들을 통째로 저장한 파일. 내 로그인 정보가 들어 있어 조심해야 합니다.
깃허브 툴	깃허브에 올라온 남이 만든 프로그램. 설치해서 내 컴퓨터에서 돌립니다.
README	깃허브 저장소의 사용 설명서. 저장소를 열면 맨 아래 보이는 긴 글.
터미널	글자로 명령을 치는 검은 창. Cursor 아래쪽에 있는 그것.
GitHub Actions	깃허브가 정해진 시각마다 내 코드를 대신 실행해주는 기능. 서버 없이 '혼자 도는 코드'를 만들 수 있습니다.
Secrets	깃허브 저장소 설정에 비밀값을 넣어두는 금고. 키와 토큰은 여기에.
환경변수	코드 밖에 적어두고 코드가 꺼내 쓰는 값. `.env.local`과 Secrets가 환경변수를 넣는 두 가지 자리입니다.
웹훅	"이 주소로 글을 보내면 채팅방에 올라간다"는 주소. 디스코드가 이 방식.
plan.md	팀과 AI가 같이 보는 설계도. 2차 때 만든 그 파일. 이번엔 '원본 해부' 섹션이 추가됩니다.

차례

1 리버스 엔지니어링 — 뜯는다

개념 · 어디까지 뜯나 · 절차 · 네트워크 탭 쓰는 법 · plan.md 양식 · 프롬프트 · 막히는 지점 · 선 지키기

2 깃허브 툴 적용하기

저장소 읽는 법 · 설치 방식 4가지 · 사용 절차 · 이번 세션 도구 카드 · 도구 대사전 · 고르는 기준 · 프롬프트

3 API 활용하기

개념 · 공개와 비공개 · 키 발급 · 호출해보기 · 키 관리 · 한도와 출처 · API 대사전 · 프롬프트 · 막히는 지점

4 텔레그램 봇으로 알림 받기

개념 · 봇 만들기 · 메시지 보내기 · 혼자 도는 코드 · 상태 기억하기 · Secrets · 다른 채널 · 막히는 지점

5 네 가지를 하나로 — 진행 순서

여덟 단계 진행 순서 · 역할 나누기 · plan.md 전체 양식 · 제출 체크리스트

부록 사전 세팅 체크리스트 · 참고 링크

1

리버스 엔지니어링 — 뜯는다

완성된 앱을 손님처럼 써보면서, 그 앱이 어떤 화면으로 어떤 값을 어디서 받아 보여주는지 설계도로 복원합니다.

이 장이 끝나면

- 리버스 엔지니어링이 뭔지, 어디까지 허용되는지 한 문장으로 말할 수 있다
- 크롬 네트워크 탭을 열어 어떤 앱이 어디서 값을 받는지 찾을 수 있다
- plan.md에 '원본 해부' 섹션을 채울 수 있다

1.1 리버스 엔지니어링이 뭔가

완성품을 보고 설계도를 거꾸로 그려내는 일입니다. 보통 만드는 순서는 설계도 → 제품이지만, 리버스 엔지니어링은 제품 → 설계도입니다. 2차 해커톤 때 plan.md를 먼저 쓰고 코드를 만들었죠. 이번엔 남의 완성된 앱을 보고 그 앱의 plan.md를 추측해서 씁니다.

> **비유 — 남의 창고에 손님으로 들어간다**
> 2차 때 plan.md를 "창고 선반 배치도"라고 했습니다. 리버스 엔지니어링은 남의 창고에 손님으로 들어가, 선반이 어떻게 놓여 있는지 보고 배치도를 그려 나오는 일입니다. 뒷문을 따지 않고, 물건을 들고 나오지 않고, 손님이 볼 수 있는 것만 봅니다.

헷갈리기 쉬운 것 하나. 리버스 엔지니어링은 **코드를 가져오는 게 아닙니다.** "이 앱은 화면이 세 개고, 두 번째 화면의 '도착까지 3분'은 어디선가 10초마다 받아오고, 저장되는 건 즐겨찾는 역 이름뿐이다" — 이런 걸 알아내는 겁니다. 알아낸 설계도로 우리 버전을 만드는 거지, 남의 코드를 복사해 오는 게 아닙니다.

왜 해커톤에서 이걸 하나

- **설계가 빨라집니다.** 백지에서 "무슨 테이블이 필요하지"를 고민하는 대신, 이미 돌아가는 앱이 답을 보여줍니다.
- **비틀 지점이 보입니다.** 원본을 뜯다 보면 "왜 이건 안 해주지?"가 자연스럽게 나옵니다. 그게 이번 해커톤의 결과물입니다.
- **AI에게 줄 재료가 생깁니다.** "지하철 알림 만들어줘"보다 "@plan.md의 원본 해부대로, 이 값이 3분 이하면 알려줘"가 훨씬 잘 됩니다.

1.2 어디까지 뜯나 — 세 겹까지, 네 번째 겹은 안 한다

앱은 양파처럼 겹이 있습니다. 바깥에서 안으로 갈수록 알아내기 어렵고, 법과 약관의 선에 가까워집니다. 이번 세션은 **세 번째 겹까지만** 합니다.

겹	무엇	어떻게 알아내나	이번 세션
1	**화면**	직접 써보며 캡처. 어떤 화면이 있고, 어떤 순서로 넘어가고, 각 화면에 무슨 값이 보이나	○ 전원
2	**값의 출처**	웹 서비스면 내 브라우저의 네트워크 탭. 폰 앱이면 공식 API 문서를 찾아 대조	○ 전원
3	**기술스택**	확장 프로그램(Wappalyzer)이 알려줌. "Supabase를 쓰는구나" 정도	○ 전원
4	앱 파일 속 코드	APK를 풀어 코드를 읽는 것. 전용 도구(jadx, Ghidra)가 필요	**× 안 함**

> **네 번째 겹을 안 하는 이유**
> 앱 파일을 여는 순간 약관 위반과 저작권 문제가 생깁니다(1.8 참고). 그리고 4시간 안에 결과가 안 나옵니다. 깃허브에서 "reverse engineering"으로 검색하면 나오는 도구 대부분이 이 네 번째 겹용이라 멋있어 보이지만, 이번 세션 범위 밖입니다.

1.3 절차 — 다섯 단계

1. **사용자로서 끝까지 써본다 — 10분**
   앱을 처음 켜는 사람처럼, 핵심 기능 하나를 끝까지 써보며 화면마다 캡처합니다. 지하철 앱이면 "역 검색 → 도착 정보 보기 → 즐겨찾기 추가"까지. 캡처는 팀 폴더 `docs/원본캡처/`에.

2. **화면 흐름을 3장 이내로 그린다**
   캡처를 보며 화면 흐름을 그립니다. 2차 때 [저장] 표시를 했듯, 이번엔 화면에 보이는 값 중 **"어디선가 받아온 값"에 [받아옴] 표시**를 합니다. "도착까지 3분" [받아옴], "역 이름" [받아옴], "즐겨찾기 ★" [저장].

3. **값의 출처를 찾는다 — 웹과 앱이 다르다**
   웹 서비스(주소창이 있는 것): 크롬에서 F12 → Network 탭 → 화면을 다시 조작 → 어떤 주소로 요청이 가는지 봅니다. 1.4에 자세히.
   폰 앱(주소창이 없는 것): 네트워크 탭을 쓸 수 없고, 쓸 필요도 없습니다. 공공데이터포털·서울열린데이터광장에서 서비스 이름으로 검색하면 공식 API 문서가 나옵니다. 3장에서.

4. **기술스택과 추정 테이블**
   Wappalyzer 아이콘을 눌러 기술스택을 캡처합니다. 그리고 [받아옴]과 [저장] 표시를 합쳐 "이 앱은 이런 테이블을 갖고 있겠다"를 추정합니다. 2차 테이블 설계 요령 그대로 — "무엇 하나당 한 줄".

5. **plan.md '원본 해부' 섹션에 정리**
   1.5의 양식대로 적고 팀 전원이 읽고 OK. 이게 끝나기 전엔 코드를 만들지 않습니다.

1.4 네트워크 탭 쓰는 법 — 웹 서비스일 때

네트워크 탭은 "지금 이 페이지가 어디에 무엇을 요청하고 있나"를 실시간으로 보여주는 창입니다. 처음 열면 수백 줄이 떠서 겁나지만, 볼 건 몇 줄뿐입니다.

1. **F12 → Network 탭**
   크롬에서 대상 페이지를 연 채 F12. 위쪽 탭에서 **Network**를 누릅니다. 창이 아래가 아니라 오른쪽에 붙어 있어도 상관없습니다.

2. **필터에서 Fetch/XHR만 켠다**
   필터 줄에 All · Fetch/XHR · Doc · CSS · JS … 가 있습니다. **Fetch/XHR**만 누릅니다. 이게 "데이터를 받아오는 요청"만 걸러줍니다. 이미지·글꼴·스타일은 전부 사라집니다.

3. **목록을 비우고, 화면을 한 번 조작한다**
   🚫 모양의 Clear 버튼으로 목록을 비운 뒤, 역을 검색하거나 새로고침 버튼을 누릅니다. **조작 직후에 새로 생긴 줄**이 그 조작이 불러온 요청입니다. 보통 1~5줄.

4. **줄을 클릭해 Headers와 Response를 본다**
   Headers 탭의 맨 위 **Request URL**이 요청 주소. Response 탭이 돌아온 데이터(대개 JSON). 여기서 화면의 "3분"이 응답의 어느 글자에서 왔는지 찾습니다. Ctrl+F로 화면에 보인 숫자를 검색하면 빠릅니다.

5. **HAR로 내보낸다 (도구에 넣을 때만)**
   목록 위 ↓ 모양 Export HAR 버튼. 저장된 .har 파일을 2장의 mitmproxy2swagger에 넣으면 API 명세가 나옵니다.

> **HAR 파일에는 내 로그인 정보가 통째로 들어 있다**
> HAR 파일은 내 쿠키·토큰까지 저장합니다. **팀 폴더나 깃허브에 절대 올리지 마세요.** 도구로 명세를 뽑은 뒤 HAR 원본은 지웁니다. 명세 파일(api.yaml)만 남깁니다.

여기서 본 주소를 그대로 쓰면 안 되나?

안 됩니다. 네트워크 탭에 보이는 주소 대부분은 **그 회사 앱 전용 비공개 창구**입니다. 설명서가 없고, 약관으로 금지하고, 예고 없이 바뀝니다. 네트워크 탭에서 할 일은 "아, 이 값은 저기서 받는구나"까지입니다. 실제 호출은 3장의 공개 API로 합니다. 비공개 주소와 공개 API가 같은 데이터를 주는 경우가 많아서, 출처를 알면 공개 API를 찾기가 쉬워집니다. 그게 네트워크 탭을 보는 이유입니다.

1.5 plan.md '원본 해부' 섹션 양식

아래를 그대로 복사해 plan.md에 넣고 채웁니다. 5장에 plan.md 전체 양식이 있습니다.

```
## 원본 해부

### 대상
- 서비스 이름:
- 왜 골랐나 (한 줄):
- 원본이 해주지 않는 일 (우리가 만들 것):

### 화면 흐름 (3장 이내)
1. [화면1 이름] — 보이는 값: ___ [받아옴], ___ [저장]
2. [화면2 이름] — 보이는 값:
3. [화면3 이름] — 보이는 값:

### 값과 출처 대응표
| 화면에 보이는 값 | 어디서 오나 (추정) | 공개 API로 대체 가능한가 |
|---|---|---|
| 도착까지 N분 | 비공개 주소 /api/arrival (네트워크 탭) | 서울열린데이터광장 실시간 도착 |
| | | |

### 기술스택 (Wappalyzer)
-

### 추정 테이블 (원본이 가지고 있을 것)
- favorites: id, station_name, created_at
-

### 우리가 더하는 규칙
- 언제:
- 무엇을:
- 어디로: 텔레그램

### 쓴 도구와 산출물
- mitmproxy2swagger → docs/api.yaml
- Wappalyzer → docs/stack.png
```

1.6 프롬프트 모음

ChatGPT나 Cursor 채팅에 그대로 붙여넣습니다. "10줄 이내", "한 가지만" 같은 제한은 답이 길어져 읽기만 하다 끝나는 걸 막기 위한 것이니 지우지 마세요.

개념 익히기

소프트웨어에서 리버스 엔지니어링이 뭔지 10줄 이내로 설명해줘. 코드를 훔치는 것과 어떻게 다른지, 일상 물건 비유 하나를 들어줘. 한국에서 법적으로 어디까지 허용되는지 3줄만.

네트워크 탭 따라 하기

크롬 개발자 도구 네트워크 탭을 열어서, 어떤 웹사이트가 화면에 보여주는 값을 어디서 받아오는지 확인하는 방법을 단계별로 알려줘. Fetch/XHR 필터가 뭔지, HAR로 내보내는 방법까지. 지금 바로 따라 할 거야.

본 주소가 공개인지 판별

내가 네트워크 탭에서 본 요청 주소가 이거야: [주소 붙여넣기]. 이게 누구나 쓸 수 있는 공개 API야, 그 회사 앱 전용 비공개 API야? 어떻게 구분해? 같은 데이터를 주는 공개 API가 있다면 어디서 찾아?

캡처로 화면 흐름 정리

@docs/원본캡처 폴더의 화면들을 보고, 화면 흐름을 3단계 이내로 정리해줘. 각 화면에 보이는 값 중 서버에서 받아온 것 같은 값에는 [받아옴], 사용자가 입력해 저장되는 값에는 [저장]을 붙여줘.

값과 출처 대응표

@docs/api.yaml 과 @docs/원본캡처 폴더를 보고, 화면에 보이는 값마다 어떤 API 응답 필드에서 오는지 표로 정리해줘. 확실하지 않은 건 '추정'이라고 표시해.

추정 테이블

@plan.md의 화면 흐름과 값 대응표를 보고, 이 원본 앱이 가지고 있을 테이블을 추정해줘. 2차 해커톤 규칙대로 — 이름은 영어 소문자와 밑줄, id와 created_at은 항상, 무엇 하나당 한 줄.

팀원에게 5분 설명 준비

내가 팀원 둘에게 5분 안에 설명해야 해. 순서를 잡아줘: 리버스 엔지니어링이 뭔지 → 오늘 뜯는 범위는 화면·값의 출처·기술스택까지고 앱 파일은 안 연다는 것 → 네트워크 탭 여는 법 → 본 주소를 그대로 쓰면 안 되는 이유. 각 항목 한 문장씩만.

1.7 막히는 지점

증상	원인과 해결
네트워크 탭에 요청이 수백 개	Fetch/XHR 필터를 안 켠 것. 켜고, Clear 한 뒤 화면을 한 번만 조작한다.

조작했는데 새 요청이 안 생김	페이지가 처음 열릴 때 데이터를 한 번에 다 받아온 경우. Clear 하지 말고 새로고침(F5) 직후 생기는 줄을 본다.
Response가 깨진 글자	압축되거나 암호화된 응답. 그 줄은 포기하고 다른 줄을 본다. 공식 API 문서를 찾는 쪽이 빠르다.
폰 앱이라 네트워크 탭이 없다	정상. 2겹은 공식 API 문서로 대신한다. 공공데이터포털에서 서비스 이름 검색.
값의 출처를 못 찾겠다	plan.md 대응표에 '출처 불명'으로 적고 넘어간다. 공개 API에 같은 데이터가 있으면 그걸 쓰면 되고, 없으면 그 값은 결과물에서 뺀다.
해부가 1시간 넘게 걸린다	너무 큰 기능을 골랐다. 화면 3장 안에 끝나는 기능 하나로 줄인다. 해부는 수단이지 결과물이 아니다.

1.8 선을 지킨다 — 쉬운 말로 푼 법과 약관

리버스 엔지니어링은 합법인 범위와 불법인 범위가 나뉩니다. 세션에서 지켜야 할 선과 그 이유를 쉬운 말로 적습니다.

선	이유
앱 파일(APK·IPA)을 열지 않는다	한국 저작권법은 프로그램을 뜯는 걸 "원리를 확인하려는 조사·연구" 목적과 "다른 프로그램과 호환시키려는" 목적으로만 좁게 허용합니다. 그리고 거의 모든 앱의 이용약관에 "리버스 엔지니어링 금지"가 적혀 있습니다. 화면을 관찰하는 건 이 어디에도 안 걸리지만, 파일을 풀어 코드를 읽는 순간 걸릴 수 있습니다.
본인 계정, 본인 화면만	정보통신망법은 "정당한 권한 없이" 남의 시스템에 들어가는 걸 금지합니다. 내 계정으로 내 화면을 보는 건 내 권한이지만, 남의 데이터를 조회하거나 로그인을 우회하면 권한 밖입니다.
읽기만, 쓰기 요청 금지	네트워크 탭에서 본 주소로 "좌석 예약", "출석 처리" 같은 쓰기 요청을 보내는 코드는 만들지 않습니다. 보는 것과 하는 것은 다릅니다.
자동 반복 요청 금지	비공개 주소를 1분마다 부르는 건 매크로입니다. 감시는 3장의 공개 API로, 발급된 한도 안에서만.
비공개 주소를 호출하지 않는다	약관 위반이고, 어느 날 막히면 결과물이 통째로 죽습니다.
타인의 이름·사진·글을 저장하지 않는다	캡처에 다른 사람 정보가 찍혔다면 가리고 쓰거나 쓰지 않습니다. 1차 해커톤 "선을 지킨다" 규칙 그대로.
원본 로고·이름을 결과물에 쓰지 않는다	상표 문제. 결과물에는 우리가 지은 이름만. "○○ 앱 비공식" 같은 표현도 피합니다.
HAR 파일과 키를 올리지 않는다	내 로그인 정보와 열쇠. atlas-prompts에 올리기 전 반드시 검사.

> **한 줄로 기억하기**
>
> **"손님이 볼 수 있는 것만 보고, 공식 창구로만 받고, 내 이름으로만 만든다."** 이 세 가지만 지키면 이번 세션에서 법적으로 문제될 일은 없습니다.

2

깃허브 툴 적용하기

깃허브에 올라온 남의 도구를 가져다 쓰는 절차는 종류가 달라도 하나입니다. 그 절차를 익히고, 이번 세션 도구와 그 너머의 도구 지도를 봅니다.

이 장이 끝나면

- 처음 보는 깃허브 저장소에서 설치 명령을 찾아 터미널에 칠 수 있다
- mitmproxy2swagger로 HAR 파일에서 API 명세를 뽑을 수 있다
- 리버스 엔지니어링 도구가 어떤 종류로 나뉘는지, 뭘 써도 되고 뭘 쓰면 안 되는지 안다

2.1 깃허브 저장소를 읽는 법

깃허브 저장소는 **"코드 + 사용 설명서"**입니다. 우리가 만든 저장소에 plan.md와 코드가 같이 있듯, 남의 저장소에도 코드와 README가 같이 있습니다. 도구를 쓸 때 코드는 볼 필요가 없고, README만 봅니다.

README에서 세 줄만 찾는다

찾을 제목	뭐가 적혀 있나
Installation / Install / Getting Started	설치 명령. 보통 한두 줄. 이 줄의 첫 단어로 설치 방식을 압니다(2.2).
Usage / Quick Start / Example	실행 명령과 예시. "이렇게 치면 이렇게 나온다".
Requirements / Prerequisites	먼저 깔려 있어야 하는 것. Python 3.10 이상, Node 22 이상 같은 것.

나머지(Features, Contributing, License, Roadmap…)는 안 읽어도 됩니다. README가 길어서 세 제목을 못 찾겠으면 Ctrl+F로 "install"을 검색하세요.

저장소 페이지에서 같이 볼 것

- **별(★) 개수** — 오른쪽 위. 많을수록 쓰는 사람이 많고 설명이 잘 돼 있습니다.
- **마지막 커밋 날짜** — 파일 목록 위 "Latest commit". 1년 넘게 조용하면 요즘 환경에서 안 돌 수 있습니다.
- **Releases** — 오른쪽. 완성된 실행파일을 내려받는 곳. 있으면 설치가 제일 쉽습니다.

2.2 설치 방식 네 가지 — 첫 단어로 구분한다

첫 단어	뜻	먼저 필요한 것	이번 세션 도구
`pip install`	파이썬으로 만든 도구	Python	mitmproxy2swagger, mitmproxy
`npm install -g` `npx`	자바스크립트 도구. npx는 설치 없이 바로 실행	Node.js	repomix, webcrack, wakaru
`git clone` 후 실행	소스를 통째로 받아 직접 돌림. 설정이 많음	Git + 해당 언어	screenshot-to-code (운영진 데모용)
Releases 다운로드	완성된 실행파일. 설치 없음	없음	jadx, ImHex (이번엔 안 씀)

크롬 확장(Wappalyzer)과 웹사이트형 도구(DeepWiki)는 설치가 아예 없습니다. 세션에서 터미널 설치가 필요한 건 **위 두 줄, pip와 npx**뿐입니다.

사전 세팅 확인 — 터미널에 세 줄

```
python --version      # 3.10 이상이면 됨
node --version        # 22 이상이면 됨
git --version         # 아무 숫자나 나오면 됨
```

OT 때 Cursor와 Git은 세팅했으니, Python과 Node만 추가 확인입니다. 없으면 python.org와 nodejs.org에서 설치합니다. **Python 설치 화면에서 "Add Python to PATH" 체크**를 빠뜨리면 터미널이 python을 못 찾습니다.

2.3 사용 절차 — 다섯 단계

1. **README에서 Installation · Usage · Requirements 세 줄을 찾는다**
2. **설치 명령 첫 단어로 방식을 판별한다**
   pip면 Python, npm·npx면 Node가 있는지 확인.
3. **터미널에 설치 명령을 그대로 친다**
   Cursor 아래 터미널에서. 복사해서 붙여넣기. 끝에 뭔가 길게 뜨다가 멈추면 성공.
4. **`도구이름 --help`로 확인한다**
   사용법이 주르륵 뜨면 설치 성공. "명령을 찾을 수 없습니다"면 2.7의 막히는 지점으로.
5. **막히면 README 전체를 AI에 붙여넣고 묻는다**
   "이 README대로 윈도우에서 설치하려는데 이 에러가 났어: [에러 붙여넣기]". 에러 메시지는 줄이지 말고 통째로.

> **윈도우에서 pip가 안 먹을 때**
>
> `pip install ...` 이 안 되면 `python -m pip install ...`, 설치는 됐는데 `mitmproxy2swagger`가 안 먹으면 `python -m mitmproxy2swagger ...`. 이 두 줄만 알면 윈도우 설치 문제의 대부분이 풀립니다.

2.4 이번 세션 도구 카드

세션에서 실제로 쓰는 도구입니다. 조건 ①("도구 1개 이상 사용, 산출물 첨부")은 이 중 하나면 충족됩니다. **mitmproxy2swagger와 Wappalyzer 둘 중 하나는 꼭** 써보세요.

mitmproxy2swagger — 네트워크 기록 → API 명세 문서

하는 일	HAR 파일을 넣으면, 어떤 주소에 어떤 입력을 주면 어떤 응답이 오는지 정리된 명세 문서(api.yaml)를 만들어 줍니다.
설치	`pip install mitmproxy2swagger`
입력	1.4에서 내보낸 .har 파일

실행	`mitmproxy2swagger -i capture.har -o api.yaml -p https://api.example.com/v1` `-p` 뒤에는 네트워크 탭에서 본 주소들의 **공통 앞부분**을 넣습니다.
두 번 돌린다	첫 실행은 주소 목록만 뽑아 각 줄 앞에 `ignore:`를 붙여둡니다. api.yaml을 열어 필요한 줄의 `ignore:`를 지우고 **같은 명령을 다시** 실행하면 그 주소들의 명세가 채워집니다.
산출물	docs/api.yaml → plan.md에 첨부
주의	`--examples` 옵션은 쓰지 않습니다. 응답 예시에 토큰과 개인정보가 섞여 들어갑니다. 실행 후 HAR 원본은 삭제.

Wappalyzer — 기술스택 확인

하는 일	이 웹사이트가 무엇으로 만들어졌는지(프레임워크, 호스팅, 분석도구, 데이터베이스) 알려줍니다.
설치	크롬 웹스토어에서 "Wappalyzer" 검색 → 추가
실행	대상 페이지에서 주소창 옆 아이콘 클릭
산출물	캡처 → docs/stack.png → plan.md에 첨부
쓰임	"Supabase랑 Vercel을 쓰는구나"를 알면 우리와 같은 도구라 설계를 추정하기 쉽습니다. 폰 앱은 확인 불가.

repomix — 저장소를 한 파일로 묶어 AI에 넘기기

하는 일	깃허브 저장소 전체를 AI가 읽기 좋은 파일 하나로 묶습니다. 원본이 오픈소스일 때 "이 서비스의 테이블 구조 설명해줘"에 쓸 재료.
설치	없음. `npx repomix@latest --remote 소유자/저장소이름`
산출물	repomix-output.xml → Cursor 채팅에 @로 첨부
주의	큰 저장소는 수 MB. `--compress`로 줄이거나 `--include "src/**"`로 범위를 좁힙니다.

DeepWiki — 오픈소스 저장소를 문서로

하는 일	공개 저장소를 자동으로 설명서로 바꿔주고, 질문창에서 "이 프로젝트 테이블 구조가 뭐야"를 물을 수 있습니다.
설치	없음. 주소창에서 `github.com/소유자/저장소`의 github를 **deepwiki**로 바꿔 접속
심화	Claude Code 터미널에서 `claude mcp add -s user -t http deepwiki https://mcp.deepwiki.com/mcp` 한 줄이면, 터미널 안에서 바로 물을 수 있습니다.
주의	공개 저장소만. 요약이 틀릴 때가 있으니 테이블 이름 같은 건 원본 코드에서 확인.

webcrack · wakaru — 압축된 화면 코드 되살리기 (심화)

하는 일	웹사이트의 자바스크립트는 한 줄로 압축돼 있어 못 읽습니다. 이 둘이 줄을 풀고 이름을 되살려 읽을 수 있게 만듭니다. "이 화면이 어떤 조건으로 어떤 주소를 부르나"를 코드로 확인할 때.
설치·실행	`npm install -g webcrack` → `webcrack bundle.js -o out/` 그 다음 `npx @wakaru/cli@latest all out/*.js`
입력	네트워크 탭 → JS 필터 → 가장 큰 파일을 열어 저장
주의 둘	되살린 코드를 결과물에 복사하지 않습니다. 읽고 설계도만 그립니다. wakaru는 반드시 `@latest` — 예전 버전에 악성 파일로 내 컴퓨터에 파일을 쓰게 하는 취약점이 있었습니다.

screenshot-to-code — 캡처 한 장 → 화면 뼈대 (운영진 데모 전용)

하는 일	화면 캡처를 넣으면 비슷한 HTML을 만들어 줍니다. 심지어 URL이나 화면 녹화 영상도 받습니다.
왜 학생은 안 하나	`git clone` 후 Python 서버와 Node 화면을 따로 띄우고, OpenAI 또는 Anthropic 키가 필요합니다. 세팅 20분, 키 비용.
대신	Cursor 채팅에 캡처를 붙이고 "이 화면 구조를 표로 정리해줘"를 치면 같은 효과를 1분에 얻습니다.

2.5 리버스 엔지니어링 도구 대사전

깃허브에는 "리버스 엔지니어링" 도구가 수천 개입니다. 범주별로 묶어 지도를 그립니다. **이번 세션 열**이 ◯인 것만 쓰고, 나머지는 "이런 것도 있다"로 알아두면 됩니다. 세션 뒤 자기 공부용 로드맵이기도 합니다.

A. 웹 트래픽 보기 — "어디에 무엇을 요청하나"

도구	한 줄	설치	세션
크롬 개발자 도구	네트워크 탭. 모든 것의 시작. 설치 없음	F12	◯
mitmproxy	내 컴퓨터를 지나는 모든 요청을 가로채 보여주는 프록시. 폰을 같은 와이파이에 물려 폰 앱 트래픽도 볼 수 있지만 인증서 설치가 필요해 이번엔 안 씀	pip	△
mitmproxy2swagger	HAR 또는 mitmproxy 기록 → API 명세	pip	◯
HTTP Toolkit	브라우저·터미널·폰 트래픽을 가로채 보는 도구. mitmproxy의 화면 있는 버전. 무료 범위로 충분	설치 프로그램	△
Chrome Recorder	개발자 도구 안의 녹화기. 클릭 순서를 기록해 자동화 코드로 내보냄	F12 → Recorder	△
Playwright codegen	브라우저 조작을 녹화해 코드로. 공개 페이지를 읽어오는 코드 만들 때	npx	△
Google HAR Analyzer	HAR 파일을 웹에서 열어 보는 뷰어. 설치 없음	웹	△

B. API 직접 불러보기 — "받은 주소를 내 손으로"

도구	한 줄	설치	세션
브라우저 주소창	GET 요청은 주소창에 넣으면 끝. 첫 호출은 반드시 이걸로	없음	○
Bruno	Postman 대체. 요청 모음을 파일로 저장해 깃허브로 팀과 공유 가능	설치 프로그램	○
Hoppscotch	웹에서 바로 쓰는 API 클라이언트. 설치 없음	웹	○
Thunder Client / REST Client	Cursor(VS Code) 확장. 에디터 안에서 요청 보내기	확장	△
HTTPie	터미널용. `http GET 주소` 한 줄	pip	△
curl	윈도우·맥 기본 내장. 터미널에서 `curl "주소"`	없음	○
Insomnia	Postman 계열 클라이언트	설치 프로그램	△

C. API 명세 읽고 쓰기

도구	한 줄	설치	세션
Swagger UI / Redoc	api.yaml 같은 명세 파일을 예쁜 문서 화면으로. mitmproxy2swagger 결과를 보기 좋게	웹/npx	△
Scalar	Swagger UI의 요즘 버전	npx	△
APIs.guru	전 세계 공개 API 명세 모음집. "이 서비스 명세 이미 있나" 확인	웹	△
openapi-generator	명세 파일 → 호출 코드 자동 생성	npx	×

D. 기술스택 알아내기

도구	한 줄	설치	세션
Wappalyzer	크롬 확장. 아이콘 클릭	확장	○
webappanalyzer	Wappalyzer의 탐지 규칙을 커뮤니티가 이어 관리하는 저장소. 직접 돌리려면 설정 필요	git clone	△
WhatRuns	Wappalyzer와 비슷한 확장	확장	△
BuiltWith	웹사이트에 주소를 넣으면 기술스택 보고서	웹	△

E. 압축된 화면 코드 되살리기

도구	한 줄	설치	세션
webcrack	난독화 해제 + 줄 풀기 + 번들 해체	npm	△
wakaru	번들 해체 + 변수 이름 되살리기. webcrack 뒤에	npx	△

js-beautify / Prettier	줄만 예쁘게 풀기. 가장 가벼움. 크롬 개발자 도구의 { } 버튼이 이것	F12	○
de4js	웹에서 바로 쓰는 난독화 해제기	웹	△
source-map 도구들	개발자가 실수로 올려둔 원본 지도(.map)가 있으면 원본 코드가 그대로 복원됨. 있을 때만	npx	△

F. 화면 → 코드

도구	한 줄	설치	세션
screenshot-to-code	캡처·URL·녹화 영상 → HTML. 키 필요	git clone	운영진
tldraw make-real	손그림 → 동작하는 화면. 키 필요	웹	△
Cursor에 캡처 붙이기	가장 쉬움. "이 화면 구조 표로 정리해줘"	없음	○

G. 코드베이스 통째로 이해하기 (원본이 오픈소스일 때)

도구	한 줄	설치	세션
DeepWiki	저장소 → 설명서 + 질문창	웹	○
repomix	저장소 → AI용 파일 하나	npx	○
gitingest	repomix의 파이썬 버전. 주소의 github를 gitingest로 바꿔도 됨	웹/pip	△
code2prompt	같은 계열, Rust로 만든 것	설치 프로그램	△
grep.app / Sourcegraph	깃허브 전체에서 코드 검색. "이 API 주소를 쓰는 다른 프로젝트"를 찾을 때	웹	△

H. 공개 페이지 읽어오기

도구	한 줄	설치	세션
Firecrawl	웹페이지 → 깔끔한 마크다운. API 없는 공개 페이지(셔틀 시간표 같은)를 감시 대상으로 만들 때	웹/pip	△
Playwright / Puppeteer	브라우저를 코드로 조종. 로그인 없는 공개 페이지만	npm/pip	△
HTTrack / wget	사이트 통째로 내려받기. 오래된 도구	설치	×
Wayback Machine	archive.org. 그 사이트의 옛날 모습. "이 기능이 언제 생겼나"	웹	△

I. 폰 앱 뜯기 (참고 — 이번 세션 범위 밖)

도구	한 줄	세션
jadx	안드로이드 앱(APK)을 자바 코드로 되돌림. 가장 유명	×

apktool	APK를 풀고 다시 묶는 도구	×
MobSF	앱 보안 자동 분석 틀	×
Frida / objection	실행 중인 앱에 끼어들어 동작을 바꿈. 보안 연구용	×
HTTP Toolkit (Android)	폰 앱 트래픽 보기. A의 그 도구	×

J. 실행파일 뜯기 (참고 — 이번 세션 범위 밖)

도구	한 줄	세션
Ghidra	미국 NSA가 만든 무료 분석 틀. 기계어를 사람이 읽는 코드로. 이 분야의 표준	×
radare2 / rizin + Cutter	터미널 중심 분석 틀. Cutter가 화면 버전	×
x64dbg	윈도우 프로그램 디버거	×
IDA Free / Binary Ninja	상용 분석 도구. 무료판 있음	×
ImHex	파일을 바이트 단위로 보는 편집기	×
Detect It Easy	이 파일이 무엇으로 만들어졌나 판별	×
Decompiler Explorer	여러 분석 도구 결과를 웹에서 나란히 비교	×
angr	프로그램을 수학적으로 분석. 연구용	×

K. 목록 모음 — "더 보고 싶을 때"

저장소	한 줄
public-apis/public-apis	무료 공개 API 1,000개 이상. 3장 대사전의 원천
awesome-reverse-engineering	모든 플랫폼 3,500개 이상 도구 모음. 이 장의 I·J 전부가 여기
awesome-reversing	책·강의·연습문제 중심 모음
awesome-android-reverse-engineering	폰 앱 쪽 모음
awesome-ai-reverse	AI(Claude·Cursor)가 전문 도구를 조종하게 해주는 연결 모음. 2.8 참고
Awesome-API-Testing-Tools	B 범주 도구 모음

2.6 도구 고르는 기준 — 세션 중 새 도구를 발견했을 때

- **마지막 커밋이 1년 안** — 아니면 요즘 환경에서 안 돌 확률이 높습니다
- **별 500개 이상** — 절대 기준은 아니지만, 이하면 README가 부실한 경우가 많습니다
- **README에 설치 명령이 한 줄** — 설치가 세 단락이면 4시간 안에 못 씁니다

- **입력과 출력이 파일** — HAR 넣으면 yaml 나오는 식. 입출력이 분명해야 plan.md에 첨부할 산출물이 생깁니다
- **앱 파일이나 실행파일을 입력으로 받으면 제외** — I·J 범주. 범위 밖

> **보안 한 줄**
>
> 깃허브 툴을 설치하는 건 **내 컴퓨터에서 남의 코드를 실행하는 일**입니다. 별 몇 개짜리 처음 보는 저장소를 pip install 하지 마세요. 2.4의 도구 목록 밖이면 멘토에게 먼저 묻습니다.

2.7 막히는 지점

증상	해결
`'pip'은(는) 내부 또는 외부 명령이 아닙니다`	Python이 PATH에 없음. `python -m pip install ...`으로. 그래도 안 되면 Python 재설치하며 "Add to PATH" 체크.
설치는 됐는데 `mitmproxy2swagger`가 안 먹음	`python -m mitmproxy2swagger ...`으로.
`npx`가 뭔가 설치하겠냐고 물음	y 누르면 됩니다. 처음 한 번만 묻습니다.
mitmproxy2swagger 결과가 텅 빔	`-p` 주소가 틀린 것. 네트워크 탭 Request URL의 앞부분을 다시 확인. `https://` 부터 넣어야 합니다.
api.yaml에 ignore:만 가득	정상. 필요한 줄의 ignore:를 지우고 같은 명령을 한 번 더.
Wappalyzer가 아무것도 안 보여줌	페이지를 새로고침 한 뒤 다시 클릭. 폰 앱 전용 서비스면 볼 웹페이지가 없는 게 정상.
DeepWiki가 "저장소를 찾을 수 없음"	비공개 저장소거나 아직 색인 안 된 것. 공개 저장소면 몇 분 뒤 다시.

2.8 AI와 묶어 쓰기 — 한 걸음 더

요즘 깃허브에는 리버스 엔지니어링 도구를 **AI가 직접 조종하게 연결해주는 것**들이 있습니다. 원리는 2.4의 DeepWiki 연결과 같습니다. 도구를 Claude Code나 Cursor에 꽂으면, 터미널에서 "이 사이트 네트워크 요청 중 데이터 받아오는 것만 정리해줘"라고 말하면 AI가 도구를 대신 돌립니다.

- **Chrome DevTools MCP** — 구글이 공식으로 만든 것. AI가 크롬 개발자 도구(네트워크 탭 포함)를 직접 봅니다. 1.4를 AI에게 시키는 셈. 심화 과제로 추천
- **DeepWiki MCP** — 2.4 참고. 가장 쉬운 시작
- IDA·Ghidra·Frida MCP — 실행파일·폰 앱 쪽. 범위 밖이지만 "이런 시대"라는 걸 알아두면 됩니다

이 묶음은 OT 때 받은 자기조사 로드맵(6-3)에 넣기 좋은 항목입니다. 해커톤이 끝난 뒤 한 번 해보세요.

2.9 프롬프트 모음

README 해석

이 깃허브 README를 읽고 세 가지만 알려줘: ① 먼저 깔려 있어야 하는 것 ② 윈도우에서 치는 설치 명령 ③ 가장 기본적인 실행 명령 하나. [README 전체 붙여넣기]

설치 오류

이 도구를 윈도우에서 설치하려고 이 명령을 쳤는데 이런 에러가 났어. 원인과 해결 명령을 알려줘. 명령: [명령] / 에러: [에러 전체 붙여넣기]

mitmproxy2swagger 사용

mitmproxy2swagger라는 깃허브 도구가 뭘 하는 건지 5줄로. 윈도우에서 pip로 설치하고 HAR 파일을 넣어 돌리는 명령을 알려줘. -p 옵션에 뭘 넣어야 하는지, 왜 두 번 돌려야 하는지도.

명세 파일 읽기

@docs/api.yaml 을 보고, 이 서비스의 API 주소를 "무엇을 받아오는 주소인지" 기준으로 한 줄씩 설명해줘. 그리고 이 중 공공데이터포털이나 서울열린데이터광장에 같은 데이터를 주는 공개 API가 있을 만한 건 어떤 거야?

도구 판별

깃허브에서 [저장소 주소]를 찾았어. 이 도구가 ① 무엇을 입력받아 무엇을 출력하는지 ② 설치 방식이 pip·npm·clone·릴리스 중 뭔지 ③ 앱 파일이나 실행파일을 뜯는 도구인지 알려줘. 마지막이 맞으면 우리 세션에선 안 쓸 거야.

팀원에게 5분 설명 준비

팀원 둘에게 5분 안에 설명할 순서: 깃허브 저장소는 코드+설명서라는 것 → README에서 세 줄 찾기 → 설치 방식 네 가지를 첫 단어로 구분 → 오늘 쓸 도구 둘(mitmproxy2swagger, Wappalyzer)이 각각 뭘 넣으면 뭘 주는지. 각 항목 한 문장씩만.

3

API 활용하기

원본 앱이 보여주던 값을, 이번엔 우리가 공식 창구에서 직접 받습니다. 키를 받고, 주소창에 넣어보고, 코드로 옮기는 순서입니다.

이 장이 끝나면

- API가 뭔지, 공개와 비공개가 어떻게 다른지 설명할 수 있다
- 공공데이터포털에서 키를 받아 브라우저 주소창에서 JSON을 볼 수 있다
- 키를 코드에 안 쓰고 환경변수로 다룰 수 있다
- 우리 학회가 쓸 수 있는 API가 어떤 것들인지 안다

3.1 API가 뭔가

API는 어떤 서비스가 **"이 주소로 이렇게 물어보면 이런 데이터를 돌려주겠다"고 열어둔 창구**입니다.

> **비유 — 같은 가게의 두 창구**
>
> 1차·2차 때 만든 웹사이트는 **사람이 보는 창구**였습니다. 예쁜 화면으로 답합니다. API는 같은 가게의 **프로그램이 보는 창구**입니다. 표 그대로 답합니다. 지하철 앱이 보여주는 "3분 후 도착"은 사람 창구고, 그 뒤에서 앱이 물어본 프로그램 창구가 API입니다. 1장에서 한 일이 바로 "이 앱은 어느 프로그램 창구에 물어보고 있나"를 찾은 겁니다.

프로그램 창구가 돌려주는 답은 대개 **JSON**이라는 형식입니다. 겁먹을 것 없습니다. 중괄호와 따옴표로 쓴 표입니다.

```
{
  "station": "신촌",
  "line": "2호선",
  "arrivals": [
    { "direction": "홍대입구 방면", "minutes": 3 },
    { "direction": "홍대입구 방면", "minutes": 9 }
  ]
}
```

이걸 표로 바꾸면 "신촌역 2호선, 홍대 방면 3분·9분"입니다. 코드에서는 `data["arrivals"][0]["minutes"]`처럼 겹겹이 꺼냅니다. 꺼내는 코드는 AI가 써주니, 우리는 **어느 글자가 우리가 쓸 값인지**만 찾으면 됩니다.

API 호출 한 번의 구성

주소	`https://api.example.com/arrival` — 어느 창구에
입력값	`?station=신촌&line=2` — 무엇을 물어보나. 주소 뒤에 ?로 붙임
키	`&serviceKey=내키` — 나라는 증명. 입력값과 같이 붙이거나, 보이지 않는 머리말(헤더)에 넣음
응답	JSON. 위의 그것

이 네 가지가 전부입니다. "API를 쓴다"는 말은 "주소에 입력값과 키를 붙여 보내고 JSON을 받는다"입니다.

3.2 공개 API와 비공개 API

	공개 API	비공개 API
누가 쓰라고	누구나. 설명서를 올리고 키를 나눠줌	그 회사 앱만
어디서 보나	공공데이터포털, 서울열린데이터광장, 각 회사 개발자 센터	1장 네트워크 탭에 보이는 주소 대부분
설명서	있음. 입력값·응답 필드·한도까지	없음. 추측만 가능
약관	쓰라고 만든 것. 출처 표시 정도만 요구	대개 금지

안정성	바뀌면 공지함	예고 없이 바뀜. 어느 날 결과물이 죽음
이번 세션	○ **이것만**	× 보기만, 호출 금지

비공개 주소와 공개 API가 같은 데이터를 주는 경우가 많습니다. 지하철 앱이 쓰는 비공개 주소와 서울열린데이터광장의 실시간 도착 API는 결국 같은 열차 정보입니다. 그래서 1장에서 출처를 확인하면 공개 API를 찾기 쉬워집니다.

3.3 키 발급 — 네 곳의 절차

이번 세션에서 쓸 API는 거의 전부 아래 네 곳에 있습니다. 전부 **가입 즉시 또는 자동승인으로 키가 나옵니다.**

공공데이터포털 (data.go.kr) — 전국 공공 API의 본점

1. 회원가입 → 로그인
2. 검색창에 서비스 이름 (예: "단기예보", "버스도착") → 결과에서 **오픈API** 탭
3. 상세 페이지에서 두 가지 확인 — **심의유형이 "자동승인"**인지, **개발계정 트래픽**이 하루 몇 건인지 (1,000 또는 10,000이 보통)
4. **활용신청** 버튼 → 활용 목적에 "학회 해커톤 학습용" → 동의 → 신청
5. 마이페이지 → 데이터활용 → Open API → 신청한 API 클릭 → **일반 인증키(Encoding)** 복사
6. 같은 페이지 아래 "참고문서"에서 활용가이드를 내려받아 예시 주소를 봅니다

> **키가 두 개 보일 때**
>
> Encoding 키와 Decoding 키가 있습니다. 브라우저 주소창에 붙일 땐 Encoding, 코드에서 라이브러리가 알아서 붙일 땐 Decoding이 맞는 경우가 많습니다. 하나가 안 되면 다른 쪽을 시도하면 됩니다.

서울열린데이터광장 (data.seoul.go.kr) — 서울 교통의 본점

1. 회원가입 → 로그인 → 상단 **인증키 신청** (또는 나의 화면 → 인증키 관리)
2. 일반 인증키 신청 — 사용 목적 적고 신청하면 **즉시 발급**
3. 주소 형식이 정해져 있습니다: `http://openapi.seoul.go.kr:8088/내키/json/서비스명/시작번호/끝번호/입력값`
4. 지하철 실시간 도착은 주소가 조금 다릅니다: `http://swopenapi.seoul.go.kr/api/subway/내키/json/realtimeStationArrival/0/5/신촌`

기상청 API허브 (apihub.kma.go.kr) — 날씨의 본점

1. 회원가입 → 로그인 → 마이페이지에서 인증키 확인. 즉시 발급, 무료
2. API 목록 → 예특보 → 단기예보. 주소 끝에 `&authKey=내키`
3. 날씨 API는 위도·경도가 아니라 **격자 번호(nx, ny)**를 씁니다. 신촌은 대략 nx=59, ny=126. 사이트에 변환표가 있습니다

같은 단기예보가 공공데이터포털에도 있습니다. 둘 다 됩니다. 공공데이터포털 쪽은 개발계정 하루 10,000건, 자동승인.

카카오디벨로퍼스 (developers.kakao.com) — 지도·장소·메시지

1. 카카오 계정으로 로그인 → 내 애플리케이션 → **애플리케이션 추가** (이름만 적으면 됨)
2. 앱 설정 → 앱 키 → **REST API 키** 복사. 즉시
3. 호출할 땐 주소에 붙이지 않고 머리말에 넣습니다: `Authorization: KakaoAK 내키`. AI에게 "카카오 로컬 API 호출 코드"라고 하면 알아서 씁니다

2026년 7월부터 카카오맵 쪽 신규 API(대중교통·도보·자전거 길찾기, 정적 지도)가 심사 없이 하루 1,000건까지 열렸습니다.

3.4 호출해보기 — 주소창에서 코드까지

1. **브라우저 주소창에 넣는다 — 반드시 손으로 한 번**
   활용가이드의 예시 주소에 내 키를 붙여 주소창에 넣고 엔터. JSON이 보이면 성공입니다. 이 한 번이 "API가 뭔지"를 몸으로 이해시킵니다. 코드부터 짜면 뭐가 틀렸는지 모릅니다.
2. **쓸 값을 찾는다**
   JSON에서 우리가 쓸 글자가 어느 이름 아래 있는지 찾습니다. Ctrl+F로 화면에서 본 숫자를 검색. 찾은 경로를 plan.md에 적습니다: `response → body → items → item[0] → arrivalTime`
3. **키를 .env.local에 넣는다**
   프로젝트 폴더에 `.env.local` 파일을 만들고 `API_KEY=여기에키` 한 줄. 이 파일은 `.gitignore`에 있어서 깃허브에 안 올라갑니다. 2차 때 Supabase 키를 넣은 그 파일입니다.
4. **AI에게 호출 코드를 시킨다**
   3.8의 프롬프트. 나오는 코드는 대략 아래 모양입니다.

```
# fetch.py — API를 한 번 부르고 쓸 값 하나를 꺼낸다
import os, requests

API_KEY = os.environ["API_KEY"]          # .env.local 또는 Secrets에서 읽음

url = "https://apis.data.go.kr/.../getArrival"
params = {
    "serviceKey": API_KEY,
    "stationName": "신촌",
    "dataType": "JSON",                   # XML 말고 JSON으로 달라
}
r = requests.get(url, params=params, timeout=10)
r.raise_for_status()                      # 실패하면 여기서 멈춘다
data = r.json()

minutes = data["response"]["body"]["items"]["item"][0]["arrivalTime"]
print(minutes)
```

터미널에서 돌릴 땐 `.env.local`을 읽게 해야 합니다. 가장 쉬운 방법은 `pip install python-dotenv` 후 코드 맨 위에 `from dotenv import load_dotenv; load_dotenv(".env.local")` 두 줄. AI에게 "python-dotenv로 .env.local 읽게 해줘"라고 하면 됩니다.

3.5 키 관리 — 세 가지 자리

어디	언제	어떻게
.env.local	내 컴퓨터에서 돌릴 때	`API_KEY=...` 한 줄. 깃허브에 안 올라감
GitHub Secrets	GitHub Actions가 돌릴 때 (4장)	저장소 Settings → Secrets and variables → Actions → New repository secret
Vercel 환경변수	배포한 사이트가 돌릴 때	Project Settings → Environment Variables. 2차 때 한 그것

> **절대 하지 않는 것**
>
> 코드 안에 `API_KEY = "abc123..."` 처럼 직접 쓰기. 깃허브에 올라가는 순간 전 세계에 공개되고, 공공데이터 포털 키는 남이 내 한도를 다 써버릴 수 있습니다. atlas-prompts에 올리기 전 검사하는 "비밀값 지운다" 규칙이 이것 때문입니다.

3.6 한도·출처 표시·라이선스

항목	내용
일일 한도	개발계정은 보통 하루 1,000 또는 10,000건. **10분마다 부르면 하루 144번**이라 1,000짜리도 넉넉합니다. 1분마다 부르면 1,440번이라 넘칩니다. 호출 간격을 plan.md에 적는 이유.
출처 표시	결과물 화면이나 README에 "데이터: 서울열린데이터광장" 한 줄. 대부분의 공공 API가 요구합니다.
공공누리 유형	공공데이터의 이용 조건. 제1유형은 출처만 표시하면 자유. 제2·3유형은 상업적 이용이나 변경에 제약. 학습용이면 어느 유형이든 괜찮지만 상세 페이지에서 확인은 합니다.
키 활성화 시간	발급 직후 몇 분에서 최대 1시간은 "SERVICE KEY IS NOT REGISTERED"가 뜰 수 있습니다. **세션 전에 미리 발급**하는 이유.
운영계정 전환	한도를 늘리려면 활용사례를 등록하고 운영계정으로 바꿉니다. 세션에선 불필요.

3.7 API 대사전 — 우리 학회가 쓸 수 있는 것

키 발급 난이도로 등급을 매겼습니다. **0등급과 1등급만** 세션 후보입니다.

등급	뜻
0	키 없음. 주소에 넣으면 바로 답이 옴
1	가입 후 즉시 발급 (자동승인)
2	승인 대기, 계좌·사업자 등 조건. 운영진이 미리 받아두면 가능
3	학생에게 사실상 불가. 세션에서 제외

교통 — 가장 체감이 크고 선례가 많은 곳

등급	API	어디서	만들 수 있는 알림 예
1	**서울 지하철 실시간 도착**	서울열린데이터광장	"내 역 다음 열차 3분 전", "막차 N분 전"
1	서울 지하철 실시간 위치	서울열린데이터광장	열차가 몇 정거장 전인지
1	서울 지하철 혼잡도	서울열린데이터광장	"이 시간 이 칸은 여유"
1	**서울 버스 실시간 도착**	서울열린데이터광장 / 공공데이터포털	정류장 도착 알림
1	경기도 버스 도착	공공데이터포털(경기도)	송도행 광역버스
1	**따릉이 실시간 대여 현황**	서울열린데이터광장	"집 앞 거치대 자전거 생기면 알림"
1	서울 주차장 실시간	서울열린데이터광장	주차 가능 알림
1	**국토부 TAGO 버스·열차**	공공데이터포털	전국 버스 도착, 시외·고속버스, KTX·SRT 시간표. 하루 10,000건
1	한국도로공사 고속도로 교통	공공데이터포털	"경부선 막히기 시작"
1	인천공항 운항 정보	공공데이터포털	항공편 지연 알림
1	한국공항공사 국내선	공공데이터포털	김포 출발 지연
1	**카카오맵 길찾기 (대중교통·도보·자전거)**	카카오디벨로퍼스	"지금 출발하면 몇 시 도착". 하루 1,000건
1	티맵 오픈API	SK 오픈API	자동차 길찾기
3	코레일 예매, 네이버 지도 실시간 길찾기	—	비공개 또는 유료

날씨·환경

등급	API	어디서	만들 수 있는 알림 예
0	**Open-Meteo**	open-meteo.com	키 없이 전 세계 날씨. 하루 10,000건. 출처 표시. 데모용으로 최적
1	**기상청 단기예보**	공공데이터포털 / 기상청 API허브	"두 시간 뒤 비", "내일 아침 영하". 활용신청 5만 건이 넘는 가장 많이 쓰인 API
1	기상청 기상특보	기상청 API허브	폭염·한파·호우 특보 알림
1	기상청 중기예보	기상청 API허브	주말 날씨
1	**에어코리아 미세먼지**	공공데이터포털	"나쁨 되면 알림". 측정소별 실시간
1	자외선·생활기상지수	공공데이터포털(기상청)	"오늘 자외선 매우 높음"

1	한강홍수통제소 수위	공공데이터포털	한강공원 통제 예보
1	서울시 한강 수온	서울열린데이터광장	변동 알림
1	산림청 산불위험지수	공공데이터포털	등산 전 확인

교육·학사

등급	API	어디서	만들 수 있는 알림 예
1	**나이스 교육정보 개방포털**	open.neis.go.kr	전국 초·중·고 급식, 학사일정, 시간표. 학회 공교육 커리큘럼과 직결
1	**천문연구원 특일정보**	공공데이터포털	공휴일·대체공휴일. "다음 연휴 D-7"
1	대학알리미	공공데이터포털	대학 공시
1	커리어넷 진로정보	공공데이터포털	학과·직업 정보
1	도서관 정보나루	data4library.kr	전국 공공도서관 소장·인기대출. "이 책 빌릴 수 있게 되면"
1	국립중앙도서관	공공데이터포털	서지 검색
0	**연세대 공지 RSS / Y-BUS 시간표 페이지 / 학식 식단 페이지**	공개 페이지	API는 아니지만 로그인 없는 공개 정보. "새 공지 올라오면", "시간표 바뀌면". 1시간 1회 이하로

생활·문화·건강

등급	API	어디서	만들 수 있는 알림 예
1	**영화진흥위 KOBIS 박스오피스**	kobis.or.kr	"1위 바뀌면", 즉시 키 발급
1	한국관광공사 TourAPI	공공데이터포털	관광지·축제·행사. "이번 주말 근처 축제"
1	문화포털 공연전시	공공데이터포털	공연 개막 알림
1	공공체육시설	공공데이터포털	근처 체육관
1	식약처 식품영양성분	공공데이터포털	식단 앱 데이터
1	농촌진흥청 레시피	공공데이터포털	"오늘 냉장고 재료로"
1	KAMIS 농수산물 가격	kamis.or.kr	"계란값 오르면"
1	응급실 실시간 가용병상 (E-Gen)	공공데이터포털	공익형
1	심평원 병원·약국 정보	공공데이터포털	"지금 문 연 약국"

1	행안부 도로명주소	juso.go.kr	주소 검색
1	소상공인 상가정보	공공데이터포털	반경 내 업소. 하루 10,000건

공공 일반·경제·통계

등급	API	어디서	만들 수 있는 알림 예
1	열린국회정보	open.assembly.go.kr	법안 발의 알림
1	법제처 국가법령정보	law.go.kr	법령 개정
1	통계청 KOSIS	kosis.kr	통계 지표
1	한국은행 ECOS	ecos.bok.or.kr	기준금리 변동
1	한국수출입은행 환율	공공데이터포털	"달러 1,400원 아래로"
1	금융위 금융공공데이터	공공데이터포털	기업 공시
1	국토부 아파트 실거래가	공공데이터포털	"우리 동네 거래 생기면". 하루 10,000건
1	경찰청·소방청 통계	공공데이터포털	지역 안전 지표
1	**공공데이터포털 전체**	data.go.kr	오픈API 1만 개 이상. "자동승인 + 개발계정 10,000"으로 걸러서 고르면 됨

금융·투자

등급	API	어디서	비고
1	토스증권 오픈API	토스증권	2026년 8월 정식 출시. 일반 고객 즉시 발급. 시세 조회는 가능하나 주문 기능은 세션에서 금지
0	업비트 시세	업비트	시세 조회는 키 없이. 주문은 금지
2	한국투자증권 KIS	한투	증권 계좌 필요
3	은행·카드·결제	—	사업자·심사. 제외

플랫폼 — 개발자 센터 가입 즉시

등급	API	어디서	만들 수 있는 것
1	**카카오 로컬 (장소·주소·좌표)**	카카오디벨로퍼스	"근처 카페", 주소 → 좌표
1	카카오톡 메시지 "나에게 보내기"	카카오디벨로퍼스	심사 없이 내 카톡으로 보내기 가능. 단 토큰이 자주 만료돼 텔레그램보다 번거로움. 심화
1	**네이버 검색 (뉴스·블로그·쇼핑)**	네이버디벨로퍼스	"내 이름 들어간 뉴스 뜨면", 하루 25,000건

1	네이버 지도 (네이버 클라우드)	ncloud.com	지도 표시·길찾기. 결제수단 등록 필요할 수 있음
1	**YouTube Data API**	구글 클라우드	"구독 채널 새 영상". 하루 10,000 단위
1	구글 캘린더·시트	구글 클라우드	"내일 일정 아침에 정리". OAuth 설정이 조금 복잡
0	**GitHub REST API**	api.github.com	키 없이 시간당 60번. "atlas-prompts에 PR 올라오면"
1	Notion API	notion.so/my-integrations	"학회 노션 페이지 바뀌면"
1	Spotify Web API	developer.spotify.com	"플레이리스트에 곡 추가되면"
1	Steam Web API	steamcommunity.com/dev	"위시리스트 게임 세일"
1	Discord API / 웹훅	discord.com/developers	발송 채널. 4장
1	Telegram Bot API	BotFather	발송 채널. 4장
1	Slack 웹훅	api.slack.com	발송 채널
3	카카오톡 알림톡·친구톡	—	사업자 인증·채널 심사. 그래서 텔레그램을 쓰는 것
3	배민·쿠팡·당근·에브리타임·인스타	—	공개 API 없음. 비공개 호출은 약관 위반

세계 무료 — 키 없이, 연습용

등급	API	뭘 주나
0	Open-Meteo	날씨·대기질·지오코딩. 위에서 설명
0	Hacker News API	기술 뉴스 순위. "1위 글 바뀌면"
0	Wikipedia API	문서 검색·요약
0	PokeAPI	포켓몬 정보. API 연습의 고전
0	Dog CEO / Cat Facts	강아지 사진, 고양이 상식. 첫 호출 연습용
0	exchangerate.host 류 환율	환율. 무료 범위 확인 필요
0	NASA APOD (DEMO_KEY)	오늘의 천문 사진
0	**ntfy.sh**	받는 게 아니라 보내는 쪽. 계정 없이 폰 푸시. 4장
0	public-apis/public-apis	이 범주 1,000개 이상 목록. 깃허브

운영진이 미리 발급해두는 공용 키 8개

세션 중 가입이나 키 활성화에 막힌 팀을 위해 운영진이 학회 계정으로 미리 받아둡니다: ① 서울 지하철 실시간 도착 ② 따릉이 ③ 기상청 단기예보 ④ 에어코리아 ⑤ 나이스 급식 ⑥ 특일정보 ⑦ KOBIS ⑧ 카카오 로컬. 필요한 팀은 멘토에게.

3.8 프롬프트 모음

개념 익히기

API가 뭔지 10줄 이내로. 웹사이트와 API의 차이를 "같은 가게의 두 창구" 같은 비유로 설명해줘. 공개 API와 어떤 회사 앱 전용 비공개 API는 어떻게 다른지 3줄.

키 발급 따라 하기

공공데이터포털에서 "[API 이름]"을 신청하고 인증키를 받는 절차를 단계별로. 받은 키를 예시 주소에 붙여 브라우저 주소창에 넣으면 뭐가 나오는지, JSON이 뭔지 5줄로.

응답에서 값 찾기

주소창에 넣었더니 이렇게 나왔어: [응답 붙여넣기]. 이 중에서 "[화면에서 본 값]"에 해당하는 필드가 뭐야? 그 값을 꺼내는 경로를 data["..."]["..."] 형태로 알려줘.

어느 포털에서 찾나

우리 팀은 [앱 이름]을 뜯어서 그 앱에 없는 알림을 만들 거야. 그 앱이 보여주는 [데이터 이름]을 공식 공개 API로 받으려면 공공데이터포털, 서울열린데이터광장, 기상청 API허브, 카카오디벨로퍼스 중 어디가 맞아? 검색할 이름도 알려줘.

호출 코드

@plan.md의 API 섹션대로, 환경변수 API_KEY를 읽어서 이 주소를 호출하고 응답에서 [필드 이름] 필드만 꺼내 출력하는 fetch.py를 만들어줘. python-dotenv로 .env.local을 읽게 하고, requests를 써.

한도 계산

이 API는 하루 호출 한도가 [숫자]건이야. [N]분마다 호출하면 하루에 몇 번이고 한도에 걸리는지 계산해줘. 걸리면 안전한 최소 간격은 몇 분이야?

팀원에게 5분 설명 준비

팀원 둘에게 5분 안에 설명할 순서: API가 뭔지 → 공개와 비공개 차이, 오늘은 공개만 → 키 받는 법 → 주소창에 넣어 JSON 보는 법 → 호출 한도. 각 항목 한 문장씩. 마지막에 "키는 .env.local에만, 코드에 직접 안 쓴다"를 꼭 넣어줘.

3.9 막히는 지점

증상	해결
SERVICE KEY IS NOT REGISTERED	발급 직후라 아직 활성화 안 됨. 최대 1시간. 공용 키로 먼저 진행.
응답이 XML로 옴 (<item> 같은 꺽쇠)	입력값에 `dataType=JSON` 또는 `_type=json`을 붙임. 문서에 어느 쪽인지 적혀 있음.
키를 붙였는데 "인증키 오류"	Encoding 키와 Decoding 키를 바꿔 시도. 주소창은 Encoding, 코드는 Decoding인 경우가 많음.
한국어 입력값이 깨짐	주소창에 한글을 직접 쓰면 됨. 코드에서는 requests가 알아서 변환. 직접 주소를 조립했다면 AI에게 "URL 인코딩 해줘".
LIMITED NUMBER OF SERVICE REQUESTS EXCEEDS	일일 한도 초과. 내일까지 막힘. 공용 키로 넘기고 호출 간격을 늘린다.
응답은 오는데 빈 목록	입력값이 틀림. 역 이름 띄어쓰기, 날짜 형식(20261103), 격자 번호 확인.
코드에서 API_KEY를 못 찾음	.env.local이 프로젝트 최상위에 있는지, 이름이 정확히 API_KEY인지, load_dotenv를 호출했는지.

4

텔레그램 봇으로 알림 받기

아무도 링크를 열지 않아도, 정해진 간격마다 혼자 확인하고, 조건에 맞을 때만 내 폰으로 보내는 조각을 만듭니다.

이 장이 끝나면

- 텔레그램 봇을 만들고 내 폰으로 메시지 한 줄을 보낼 수 있다
- GitHub Actions로 10분마다 코드가 혼자 돌게 할 수 있다
- 같은 알림이 반복되지 않게 "마지막 상태"를 기억시킬 수 있다

4.1 "혼자 움직이는 코드"가 뭔가

지금까지 만든 건 전부 **누군가 링크를 열어야 돌아가는 것**이었습니다.

회차	언제 코드가 실행되나	비유
1차	사용자가 링크를 열 때	자판기 — 버튼을 눌러야 나온다
2차	사용자가 버튼을 눌러 저장할 때	장부 달린 자판기 — 눌러야 기록된다
3차	**아무도 안 봐도, 정해진 시각마다**	**점원 — 손님이 없어도 10분마다 창밖을 본다**

"원본 앱이 해주지 않는 일" 대부분이 세 번째 종류입니다. 지하철 앱은 내가 열어야 다음 열차를 보여주지만, "지금 나가야 해"는 내가 안 열었을 때 알려줘야 의미가 있습니다.

점원에게 필요한 네 조각

조각	역할	구현
시계	정해진 간격에 실행	GitHub Actions 스케줄 — `.github/workflows/watch.yml`
눈	API 호출, 조건 판정	`watch.py` 30줄
입	발송	텔레그램 봇 — HTTP 요청 1번
수첩	마지막에 본 값 기억	`state.json` 파일 (또는 Supabase)

수첩이 왜 필요한지는 4.5에서. 먼저 가장 쉬운 '입'부터 만듭니다.

4.2 텔레그램 봇 만들기 — 5분

1. **텔레그램 설치, 가입**
   폰에 텔레그램 앱을 설치하고 전화번호로 가입합니다. PC 버전도 있지만 폰이면 충분합니다.
2. **BotFather를 찾는다**
   검색창에 **BotFather**. 파란 체크가 붙은 공식 계정입니다. 대화를 엽니다.
3. **`/newbot` 을 보낸다**
   봇 이름을 물으면 아무거나 (예: 신촌 열차 알림). 그 다음 아이디를 물으면 **영문으로, 끝이 bot** (예: `sinchon_train_bot`). 이미 있는 아이디면 다른 걸로.
4. **토큰을 받는다**
   BotFather가 긴 글자를 줍니다. `7123456789:AAH...` 모양. 이게 **봇 토큰**입니다. 복사해서 `.env.local` 에 `BOT_TOKEN=...` 으로. 채팅방에 그대로 두지 마세요.
5. **내 봇에게 아무 말이나 보낸다**
   BotFather가 준 링크(t.me/내봇아이디)를 눌러 봇 대화방을 열고 **Start**를 누른 뒤 "안녕" 한 마디. 이게 없으면 봇이 나한테 말을 걸 수 없습니다.

6 **내 채팅 ID를 알아낸다**
브라우저 주소창에 아래를 넣습니다. (토큰 자리에 내 토큰)
`https://api.telegram.org/bot여기에토큰/getUpdates`
JSON이 뜨고 그 안에 `"chat":{"id":123456789,` 가 보입니다. 그 숫자가 **채팅 ID**. `.env.local`에 `CHAT_ID=...`으로.

> **getUpdates에 아무것도 안 나올 때**
> 5번을 안 한 것. 봇에게 메시지를 먼저 보내고 새로고침. 그래도 비어 있으면 `"result":[]` 상태이니 메시지를 한 번 더 보냅니다.

4.3 메시지 보내기 — 한 줄

텔레그램 봇으로 메시지를 보내는 건 **주소 하나에 요청 한 번**입니다. 먼저 주소창에서 확인합니다.

```
https://api.telegram.org/bot여기에토큰/sendMessage?chat_id=여기에채팅ID&text=테스트
```

엔터를 치면 폰에 "테스트"가 옵니다. 이걸로 봇은 끝났습니다. 3장에서 API를 주소창에 넣어본 것과 완전히 같은 원리입니다 — 텔레그램 봇도 그냥 공개 API입니다.

코드로는 이렇게 됩니다.

```
# send.py — 텔레그램으로 한 줄 보내기
import os, requests

BOT_TOKEN = os.environ["BOT_TOKEN"]
CHAT_ID = os.environ["CHAT_ID"]

def send(text):
    url = f"https://api.telegram.org/bot{BOT_TOKEN}/sendMessage"
    requests.post(url, data={"chat_id": CHAT_ID, "text": text}, timeout=10)

send("봇이 살아 있습니다")
```

터미널에서 `python send.py` → 폰에 도착하면 '입'은 완성입니다.

4.4 혼자 돌게 하기 — GitHub Actions

이제 '시계'입니다. 내 컴퓨터를 켜두지 않아도 되는 방법이 필요합니다. **GitHub Actions**는 깃허브가 자기 컴퓨터에서 정해진 시각마다 내 코드를 대신 실행해주는 기능입니다. 서버를 빌릴 필요가 없고, 학회가 이미 깃허브를 쓰니 새 가입도 없습니다.

저장소에 파일 하나

저장소 안에 `.github/workflows/watch.yml` 경로로 파일을 만듭니다. 폴더 이름의 점과 철자가 정확해야 합니다.

```yaml
name: watch

on:
  schedule:
    - cron: "*/10 * * * *"      # 10분마다
  workflow_dispatch:            # Actions 탭에서 수동 실행 버튼

permissions:
  contents: write               # state.json을 저장소에 쓰기 위해

jobs:
  watch:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install requests
      - run: python watch.py
        env:
          API_KEY: ${{ secrets.API_KEY }}
          BOT_TOKEN: ${{ secrets.BOT_TOKEN }}
          CHAT_ID: ${{ secrets.CHAT_ID }}
      - name: 마지막 상태 저장
        run: |
          git config user.name "watch-bot"
          git config user.email "watch-bot@users.noreply.github.com"
          git add state.json
          git diff --cached --quiet || git commit -m "update state"
          git push
```

각 줄이 하는 일

줄	뜻
`cron: "*/10 * * * *"`	10분마다. 다섯 칸은 "분 시 일 월 요일". `*/10`은 10분 간격. `0 8 * * *`면 매일 8시 — 단 **UTC 기준**이라 한국 시간 8시는 `0 23 * * *`(전날 23시 UTC)로 적어야 합니다.
`workflow_dispatch`	Actions 탭에 "Run workflow" 버튼이 생깁니다. 테스트할 때 이걸로 즉시 실행.
`permissions: contents: write`	이게 없으면 마지막 단계의 git push가 거부됩니다.
`checkout` / `setup-python`	내 저장소를 가져오고 파이썬을 준비. 남이 만든 조각을 가져다 쓰는 것. 2장의 깃허브 툴과 같은 원리.
`env:` 아래 세 줄	Secrets 금고에서 꺼내 환경변수로 넣어줌. watch.py의 `os.environ["API_KEY"]`가 여기서 값을 받습니다.
마지막 상태 저장	watch.py가 고친 state.json을 저장소에 다시 올림. 바뀐 게 없으면 commit을 건너뜀.

알아둘 제약 둘

① 최소 간격은 **5분**. 더 짧게 적어도 5분으로 처리됩니다.

② 무료라서 **몇 분 밀릴 수 있습니다.** "10:00 정각에 보낸다"로 설계하면 어긋납니다. "10분마다 확인해서 조건에 맞으면 보낸다"로 설계하면 문제없습니다.

4.5 수첩 — 같은 알림이 열 번 오지 않게

GitHub Actions는 **매번 깨끗한 새 컴퓨터**에서 시작합니다. 10분 전에 뭘 봤는지 기억이 없습니다. 그러면 두 가지 문제가 생깁니다.

- 따릉이가 1대 남은 상태가 1시간 지속되면, 10분마다 "1대 남았어요"를 6번 보냅니다
- "값이 바뀌었을 때만"이라는 규칙 자체를 못 씁니다. 바뀌었는지 알려면 이전 값이 있어야 하니까요

그래서 **'마지막으로 본 값'을 어딘가 적어둬야 합니다.** 가장 쉬운 방법은 저장소 안에 `state.json` 파일을 두고, 실행 때마다 읽고 → 고치고 → 다시 올리는 겁니다. 4.4의 마지막 단계가 그 "다시 올리기"입니다.

watch.py 전체 — 네 조각이 모인 곳

```
# watch.py — 10분마다 실행된다. 읽고, 받고, 판단하고, 보내고, 적는다.
import os, json, requests

API_KEY   = os.environ["API_KEY"]
BOT_TOKEN = os.environ["BOT_TOKEN"]
CHAT_ID   = os.environ["CHAT_ID"]
STATE_FILE = "state.json"

# 수첩 읽기 ------------------------------------------------
def load_state():
    try:
        with open(STATE_FILE, encoding="utf-8") as f:
            return json.load(f)
    except FileNotFoundError:
        return {}                          # 첫 실행이면 빈 수첩

def save_state(state):
    with open(STATE_FILE, "w", encoding="utf-8") as f:
        json.dump(state, f, ensure_ascii=False, indent=2)

# 눈: API 호출 ---------------------------------------------
def fetch_now():
    url = "https://여기에_plan.md의_API_주소"
    params = {"serviceKey": API_KEY, "dataType": "JSON",
              "stationName": "신촌"}       # plan.md의 입력값
    r = requests.get(url, params=params, timeout=10)
    r.raise_for_status()
    data = r.json()
    # plan.md에 적어둔 경로대로 쓸 값 하나만 꺼낸다
    item = data["response"]["body"]["items"]["item"][0]
    return {"train": item["trainNo"], "minutes": int(item["arrivalTime"])}

# 입: 텔레그램 ---------------------------------------------
def send(text):
    url = f"https://api.telegram.org/bot{BOT_TOKEN}/sendMessage"
    requests.post(url, data={"chat_id": CHAT_ID, "text": text}, timeout=10)

# 판단 ----------------------------------------------------
state = load_state()
now = fetch_now()

should_send = now["minutes"] <= 3                      # plan.md의 규칙
already_sent = state.get("last_sent_train") == now["train"]

if should_send and not already_sent:
    send(f"[열차] 신촌역 열차 {now['minutes']}분 뒤 도착. 지금 나가세요.")
    state["last_sent_train"] = now["train"]           # 수첩에 적는다

state["last_seen"] = now
save_state(state)
```

구조만 기억하면 됩니다. **읽고(load) → 받고(fetch) → 판단하고(if) → 보내고(send) → 적는다(save).** 조건 한 줄(`should_send`)과 "이미 보냈나" 한 줄(`already_sent`)만 팀의 plan.md대로 바꾸면 어떤 알림이든 됩니다.

Supabase로 수첩을 쓰고 싶다면 (가산)

state.json 대신 2차 때 만든 Supabase 테이블에 적어도 됩니다. 테이블은 세 개면 됩니다 — `watch_targets` (무엇을 보나), `last_states` (마지막에 본 값), `sent_logs` (언제 뭘 보냈나). 장점은 여러 사람이 설정 화면에서 감시 대상을 바꿀 수 있다는 것. 단점은 세팅 30분이 더 든다는 것. **세션에서는 state.json으로 먼저 끝내고, 시간이 남으면 옮기세요.**

4.6 Secrets 등록 — 금고에 넣기

1. 깃허브 저장소 페이지 → **Settings** (저장소 설정, 내 계정 설정 아님)
2. 왼쪽 메뉴 → Secrets and variables → **Actions**
3. **New repository secret** → Name에 `API_KEY`, Secret에 키 값 → Add secret
4. `BOT_TOKEN`, `CHAT_ID` 도 같은 방식으로. 이름은 watch.yml의 `secrets.이름` 과 글자까지 같아야 합니다
5. 한 번 넣으면 값을 다시 볼 수 없습니다. 정상입니다. 잘못 넣었으면 Update로 덮어씁니다

4.7 처음부터 끝까지 — 10분 안에 첫 알림

1. **운영진 데모 저장소를 포크한다**
   watch.yml · watch.py · state.json · README가 들어 있습니다. 포크하면 내 계정에 복사본이 생깁니다.
2. **Actions를 켠다**
   포크 직후엔 꺼져 있습니다. 저장소의 **Actions** 탭 → "I understand my workflows, go ahead and enable them".
3. **Secrets 세 개를 넣는다**
   4.6대로.
4. **watch.py의 조건 부분만 고친다**
   `fetch_now`의 주소·입력값·꺼내는 경로, `should_send` 한 줄. 4.9의 프롬프트로 AI에게.
5. **수동 실행으로 확인**
   Actions 탭 → 왼쪽 watch → **Run workflow** → 초록 체크가 되면 폰 확인. 빨간 X면 클릭해서 어느 줄에서 멈췄는지 봅니다.
6. **10분 뒤, 같은 알림이 안 오는지 확인**
   이게 "같은 알림 반복 금지" 조건의 검사입니다. 또 오면 4.5의 수첩이 안 돌고 있는 것.

4.8 다른 발송 채널

텔레그램이 기본이지만, 팀 사정에 따라 아래로 바꿔도 조건 ③은 충족됩니다. 코드에서 바뀌는 건 `send` 함수 안의 주소 한 줄뿐입니다.

채널	준비	장단점

텔레그램	4.2 — 봇 토큰 + 채팅 ID	5분. 폰 앱 설치 필요. 가장 안정적
디스코드	채널 설정(톱니) → 연동 → 웹훅 → 새 웹훅 → 웹훅 URL 복사. 코드: `requests.post(WEBHOOK_URL, json={"content": text})`	1분. 토큰 없음. 이미 쓰는 학생 많음. 운영진 데모용
ntfy	폰에 ntfy 앱 설치 → 아무 이름으로 구독(예: atlas-beomjun-7f3k) → 코드: `requests.post("https://ntfy.sh/그이름", data=text.encode())`	2분. 계정조차 없음. 이름을 남이 맞추면 그 사람도 받으니 긴 이름으로
슬랙	Incoming Webhook 앱 추가 → URL	학회에서 안 쓰면 불필요
카카오톡 나에게 보내기	카카오디벨로퍼스 메시지 API. 심사는 없지만 토큰이 몇 시간마다 만료돼 갱신 코드가 필요	심화. 세션에선 비추천
카카오톡 알림톡	—	사업자 인증 필요. 불가

4.9 프롬프트 모음

개념 익히기

"아무도 웹사이트를 열지 않아도 정해진 시간마다 혼자 돌아가는 코드"가 뭔지 10줄 이내로. 사용자가 버튼을 눌러야 돌아가는 코드와 뭐가 다른지 비유 하나로.

텔레그램 봇

텔레그램 BotFather로 봇을 만들고, 봇 토큰과 내 채팅 ID를 알아내서, 파이썬 requests로 내 폰에 메시지 한 줄 보내는 가장 짧은 코드를 알려줘. 토큰은 환경변수 BOT_TOKEN에서 읽게 해.

GitHub Actions

GitHub Actions가 뭔지 5줄로. 10분마다 watch.py를 실행하고 state.json을 커밋하는 workflow 파일을 보여주고, 각 줄이 무슨 뜻인지 한 줄씩. cron이 UTC라는 것과 한국 시간으로 바꾸는 법도.

수첩이 왜 필요한가

10분마다 API를 확인해서 조건에 맞으면 알림을 보내는 코드를 만들 거야. 그런데 조건이 1시간 동안 계속 맞으면 같은 알림이 6번 오잖아. 이걸 막으려면 뭘 어디에 기억해둬야 해? GitHub Actions가 매번 깨끗한 컴퓨터에서 시작한다는 점을 포함해서 설명해줘.

watch.py 수정

@plan.md의 알림 규칙대로 @watch.py를 고쳐줘. fetch_now는 [API 주소]를 [입력값]으로 호출해서 [꺼낼 경로]의 값을 돌려주고, 조건은 "[조건]"이야. state.json의 last_sent와 같은 [기준]이면 보내지 마. 발송은 텔레그램 그대로.

실행 실패 읽기

GitHub Actions 실행이 빨간 X로 끝났어. 로그 이거야: [로그 붙여넣기]. 어느 단계에서 왜 멈췄고 뭘 고치면 돼?

팀원에게 5분 설명 준비

팀원 둘에게 5분 안에 설명할 순서: 혼자 돌아가는 코드가 뭔지 → GitHub Actions가 그걸 대신 돌려준다 → 시계·눈·입·수첩 네 조각 → 수첩이 없으면 같은 알림이 반복된다 → Secrets. 각 항목 한 문장씩.

4.10 막히는 지점

증상	해결
getUpdates에 아무것도 없음	봇에게 먼저 메시지를 보내지 않았다. Start 누르고 한 마디 보낸 뒤 새로고침.
sendMessage가 "chat not found"	채팅 ID가 틀렸거나 봇과 대화를 시작하지 않았다. getUpdates의 id를 다시 확인. 음수(-100...)면 그룹 ID이니 그대로 쓴다.
"Unauthorized"	봇 토큰이 틀림. 콜론 앞뒤 전체를 복사했는지 확인.
스케줄이 안 돈다	① 포크 직후 Actions 꺼짐 → 켠다 ② 파일이 main 브랜치에 없음 → main에 있어야 스케줄이 돈다 ③ 폴더 이름이 `.github/workflows` 가 아님 ④ 60일간 저장소에 활동이 없으면 깃허브가 스케줄을 끈다 → Actions 탭에서 다시 켠다.
수동 실행은 되는데 스케줄은 늦음	정상. 몇 분 밀릴 수 있다. "N분마다"로 설계했으면 문제없다.
알림이 10분마다 똑같이 온다	수첩을 안 읽거나 안 쓰고 있다. ① state.json 커밋 단계가 있는지 ② `permissions: contents: write` 가 있는지 ③ already_sent 비교 기준이 맞는지.
git push가 거부됨 (403)	`permissions: contents: write` 누락. 또는 저장소 Settings → Actions → General → Workflow permissions가 "Read and write"인지.
KeyError: 'API_KEY'	Secrets 이름이 watch.yml의 env 이름과 다르다. 대소문자까지 같아야 한다.
토큰을 코드에 직접 썼다	지금 바로 지우고, BotFather에게 `/revoke` 로 토큰을 새로 받는다. 깃허브에 한 번 올라간 토큰은 지워도 기록에 남는다.

5

네 가지를 하나로 — 진행 순서

앞의 네 장을 하나의 순서로 잇습니다. 앞 단계가 끝나야 다음 단계의 재료가 생기므로, 순서를 건너뛰지 않습니다. 역할을 나누고, plan.md 전체 양식을 채우고, 제출 전 체크리스트를 돕니다.

5.1 진행 순서 — 여덟 단계

각 단계의 핵심은 **"끝났다"의 기준**입니다. 그 기준을 넘기 전에 다음 단계로 가면 뒤에서 되돌아오게 됩니다. 소요시간은 3인 팀 기준의 대략적인 감입니다.

1. **개념 잡기 — 20~25분**
   키워드 5개(리버스 엔지니어링·깃허브 툴·공개 API·텔레그램 봇·GitHub Actions)를 각자 검색. **끝의 기준:** "앱을 뜯어서, 공개 API로 받아서, 깃허브가 혼자 돌리다, 텔레그램으로 알린다"를 자기 말로 한 문장씩 말할 수 있다.

2. **운영진 데모 보기 — 10분**
   포크 → Secrets → 수동 실행 → 폰에 첫 알림. **끝의 기준:** 마지막에 내가 만들 게 어떤 모양인지 눈으로 봤다.

3. **대상 고르기와 해부 — 30~40분 (1장·2장)**
   서비스 하나를 골라 손님처럼 써보고, 화면 3장과 값의 출처를 찾는다. 팀장은 캡처와 HAR, 팀원은 Wappalyzer와 공개 API 검색. **끝의 기준:** plan.md '원본 해부' 섹션이 채워졌고 팀 전원이 읽고 OK했다. **이 전엔 코드를 만들지 않는다.**

4. **API 손으로 확인 — 15분 (3장)**
   키를 받아 브라우저 주소창에 넣고 JSON을 본다. 쓸 값의 경로를 plan.md에 적는다. **끝의 기준:** 주소창에서 우리가 쓸 값이 보였고, 경로가 plan.md에 적혀 있다.

5. **뼈대 세우기 — 15분 (4장)**
   팀장이 데모 저장소를 포크하고 Secrets 세 개를 넣어 뼈대 PR을 올린다. 팀원은 봇 토큰과 채팅 ID를 준비. **끝의 기준:** 수동 실행에서 "봇이 살아 있습니다"가 폰에 왔다.

6. **구현 — 가장 긴 구간, 60분 안팎**
   각자 브랜치에서. watch.py의 fetch_now와 조건 한 줄, 설정 화면, 메시지 문구. AI에게는 "@plan.md대로"로 시킨다. **끝의 기준:** 수동 실행으로 진짜 알림이 왔다.

7. **반복 검사 — 10분 기다림**
   스케줄이 한 번 더 돌 때까지 기다려 같은 알림이 안 오는지 본다. 오면 수첩(state.json)이 안 돌고 있는 것. **끝의 기준:** 두 번째 실행에서 알림이 안 왔고, 저장소에 "update state" 커밋이 생겼다.

8. **마무리 — 20분**
   배포 URL, README 출처 표시, 키·HAR 검사, atlas-prompts 정리, 1분 발표 준비("원본은 안 해주고, 우리는 해준다"). **끝의 기준:** 5.4 체크리스트 전부 ✓.

> **순서를 지키는 이유**
> 3이 끝나야 4에서 뭘 받을지 알고, 4가 끝나야 6에서 조건을 쓸 수 있고, 5가 끝나야 6의 결과를 폰에서 볼 수 있습니다. 시간이 모자라면 **6의 설정 화면을 줄이지, 3·4·5를 건너뛰지 않습니다.** 설정 화면 없는 알림 봇은 완성이지만, 해부 없는 설정 화면은 복제입니다.

5.2 역할 나누기 — 3인 팀 기준

역할	공부하는 장	세션 중 맡는 것
뜯는 사람	1장 + 2장	원본 캡처, 네트워크 탭, mitmproxy2swagger, plan.md 원본 해부 섹션

받는 사람	3장	API 검색·키 발급, 주소창 테스트, 쓸 값 경로, fetch_now 함수
알리는 사람	4장	봇 만들기, 데모 저장소 포크, Secrets, watch.yml, 반복 방지 확인

팀장은 셋 중 하나를 겸하면서 저장소와 PR을 관리합니다. 4명이면 "뜯는 사람"을 둘로 나눠 한 명이 2장 도구를 전담합니다.

5.3 plan.md 전체 양식

```
# [우리 서비스 이름]

## 한 줄
[원본 앱]은 [무엇]을 안 해준다. 우리는 [무엇]을 텔레그램으로 알려준다.

## 원본 해부
(1장 1.5의 양식 그대로)

## 데이터 (공개 API)
- API 이름:
- 어디서: 공공데이터포털 / 서울열린데이터광장 / 기상청 API허브 / 카카오
- 주소:
- 입력값:
- 쓸 값의 경로: data["..."]["..."]
- 일일 한도:
- 우리 호출 간격:      분마다 → 하루     번
- 출처 표시 문구: "데이터: ○○○"

## 알림 규칙
- 확인 간격: 10분
- 보내는 조건:
- 같은 알림 반복 방지 기준: (예: 같은 열차 번호면 안 보냄)
- 메시지 문구:
- 채널: 텔레그램

## 수첩 (마지막 상태)
- 저장 위치: state.json
- 적는 값: last_sent_○○, last_seen

## 설정 화면 (배포 URL)
- 화면 1장: 무엇을 보여주나

## 비틀기
- 원본에 없는 것:

## 선 지키기 확인
- [ ] 앱 파일 안 열었다
- [ ] 공개 API만 쓴다
- [ ] 네트워크 탭은 보기만 했다
- [ ] HAR 원본 삭제했다
- [ ] 키·토큰은 Secrets와 .env.local에만
- [ ] 원본 로고·이름 안 쓴다

## 쓴 도구와 산출물
-

## 역할
- 뜯기: / 받기: / 알리기:
```

5.4 제출 체크리스트

✓	항목	조건
☐	plan.md에 '원본 해부' 섹션이 있고, 도구 산출물(api.yaml 또는 stack.png)이 첨부돼 있다	①
☐	데이터가 공식 공개 API 또는 공개 페이지에서 온다. plan.md에 API 이름·한도·간격이 적혀 있다	②
☐	GitHub Actions 스케줄이 켜져 있고, Actions 탭에 초록 체크 기록이 있다	③
☐	텔레그램(또는 대체 채널)에 실제 알림이 도착한 캡처가 있다	③
☐	10분 뒤 같은 알림이 반복되지 않았다 (state.json이 커밋되고 있다)	③
☐	원본에 없는 기능 하나가 있고, 발표에서 "원본은 안 해주고 우리는 해준다"를 1분 안에 보여준다	④
☐	배포 URL이 있다 (설정 화면 하나면 충분)	기존
☐	코드·README·atlas-prompts 어디에도 키·토큰·HAR 파일이 없다	⑤
☐	README에 데이터 출처가 적혀 있고, 원본 로고·이름을 쓰지 않았다	⑤
☐	브랜치·PR 규칙대로 합쳤고, 각자 prompts/내이름/에 이번 회차 대화를 올렸다	협업

5.5 평가 기준

항목	보는 것
해부 정확도	plan.md의 원본 분석이 실제 원본과 맞는가. 멘토가 원본을 열어 대조
작동	세션 끝에 실제로 알림이 왔는가. 반복되지 않는가
설득력	"원본이 왜 이걸 안 해줬을까"에 대한 팀의 답. 비틀기가 쓸모 있는가
선	체크리스트 ⑤ 항목. 하나라도 어기면 결과 점수와 무관하게 감점

부록

A. 사전 세팅 체크리스트

✓	항목	확인 방법
☐	Python 3.10 이상	터미널 `python --version`. 설치 시 "Add to PATH" 체크
☐	Node.js 22 이상	터미널 `node --version`
☐	Git	터미널 `git --version`. OT 때 완료
☐	mitmproxy2swagger	`pip install mitmproxy2swagger` → `mitmproxy2swagger --help`
☐	requests, python-dotenv	`pip install requests python-dotenv`
☐	Wappalyzer 확장	크롬 웹스토어 → 주소창 옆 아이콘 보임
☐	공공데이터포털 계정	가입 후 아무 API 하나 활용신청까지 (키 활성화 시간 때문)
☐	서울열린데이터광장 계정 + 일반 인증키	가입 → 인증키 신청 → 즉시 발급 확인
☐	텔레그램 설치 + 봇 1개 + 채팅 ID	4.2 완료, 주소창 sendMessage로 폰에 "테스트" 도착
☐	깃허브 Secrets 위치 확인	아무 저장소 Settings → Secrets and variables → Actions 열어보기

윈도우에서 pip가 안 먹으면 `python -m pip install ...`, 도구가 안 먹으면 `python -m 도구이름 ...`.

B. 참고 링크

이름	주소
공공데이터포털	data.go.kr
서울열린데이터광장	data.seoul.go.kr
기상청 API허브	apihub.kma.go.kr
나이스 교육정보 개방포털	open.neis.go.kr
카카오디벨로퍼스	developers.kakao.com
네이버디벨로퍼스	developers.naver.com
텔레그램 봇 API 문서	core.telegram.org/bots/api
GitHub Actions 스케줄 문서	docs.github.com → Actions → Events → schedule

mitmproxy2swagger	github.com/alufers/mitmproxy2swagger
repomix	github.com/yamadashy/repomix
DeepWiki	deepwiki.com
webcrack / wakaru	github.com/j4k0xb/webcrack · github.com/pionxzh/wakaru
public-apis	github.com/public-apis/public-apis
awesome-reverse-engineering	github.com/alphaSeclab/awesome-reverse-engineering
Open-Meteo	open-meteo.com
ntfy	ntfy.sh

C. 한 장 요약

뜯는다

손님처럼 써보고 → 화면 3장 → [받아옴] 표시 → 네트워크 탭은 보기만 → plan.md 원본 해부. 앱 파일은 안 연다.

도구로 뜯는다

README에서 Installation·Usage·Requirements 세 줄 → 첫 단어로 pip/npx 판별 → 터미널에 그대로 → --help로 확인. HAR → mitmproxy2swagger → api.yaml.

받는다

포털에서 자동승인 확인 → 키 발급 → 주소창에 넣어 JSON 확인 → 쓸 값 경로 적기 → 키는 .env.local과 Secrets에만. 10분마다면 하루 144번.

알린다

BotFather → 토큰 → getUpdates로 채팅 ID → sendMessage 주소창 테스트 → 데모 저장소 포크 → Actions 켜기 → Secrets 3개 → watch.py 조건 수정 → Run workflow → 10분 뒤 반복 안 오는지.

선

본인 계정·본인 화면만 · 읽기만 · 공개 API만 · 앱 파일 안 열기 · HAR·키·토큰 안 올리기 · 원본 로고 안 쓰기.